Predicting Sports Betting Outcomes Using a GNN

Kishan Peddi
Northern Illinois University
CSCI

Jerry Guerrero
Northern Illinois University
CSCI

Nick Loisi
Northern Illinois University
CSCI

I. Motivation

Sports betting represents a multi-billion dollar global industry that depends on accurate predictions of game outcomes. Traditional ML models for sports analytics usually treat each game as an isolated event, not taking into account the network of relationships among teams, players, and historical matches.

Our goal is to explore whether a Graph Neural Network (GNN) can effectively predict game outcomes (win/loss or score difference) based on relational structures derived from historical sports data. This project is motivated by our curiosity and the potential application of graph-based models in the sports analytics and betting industries.

II. Task Definition

Our **Data source** will be mainly derived from free, public data sets of NFLs historical data (NFLverse, NFLsavant, Australia Sports Betting, etc) [4]. These repositories provide the necessary **raw data** in CSV format, or spreadsheets, making them easy to read and analyze (https://www.aussportsbetting.com/data/historical-nfl-results-and-odds-data/).

Date	Home Team	Away Team	Home Score	Away Score	Overtime?	Playoff Game?	Neutral Venue?	Home Odds Open	Home Odds Min
2025-10-20	Seattle Seahawks	Houston Texans	27	19				1.88	1.53
2025-10-20	Detroit Lions	Tampa Bay Buccaneers	24	9				1.51	1.35
2025-10-19	San Francisco 49ers	Atlanta Falcons	20	10				1.35	1.35
2025-10-19	Arizona Cardinals	Green Bay Packers	23	27				2.00	1.99
2025-10-19	Dallas Cowboys	Washington Commanders	44	22				2.10	1.79
2025-10-19	Denver Broncos	New York Giants	33	32				1.31	1.23
2025-10-19	Los Angeles Chargers	Indianapolis Colts	24	38				1.39	1.38
2025-10-19	Chicago Bears	New Orleans Saints	26	14				1.26	1.26
2025-10-19	Cleveland Browns	Miami Dolphins	31	6				1.98	1.62
2025-10-19	Kansas City Chiefs	Las Vegas Raiders	31	0				1.23	1.10
2025-10-19	Minnesota Vikings	Philadelphia Eagles	22	28				2.30	2.05
2025-10-19	New York Jets	Carolina Panthers	6	13				1.83	1.68
2025-10-19	Tennessee Titans	New England Patriots	13	31				2.10	2.10
2025-10-18	Jacksonville Jaguars	Los Angeles Rams	7	35				2.65	2.25
2025-10-16	Cincinnati Bengals	Pittsburgh Steelers	33	31				1.47	1.44

Fig. 1. Australia Sports Betting NFL Data

The goal of this project is to predict the outcome of NFL games using a supervised learning framework on a GNN. We model each game as a directed edge connecting two teams, and the prediction task is formulated as **edge-level binary classification**. For every game, the model should output the probability that the **home team wins**.

The input to the prediction problem consists of:

- the home team identity
- the away team identity
- pre-game betting market odds for both teams
- the historical graph structure of past games between all teams

The label for each game is defined as:

- **1** if the home team won
- **0** if the away team won

This setup allows the model to leverage the relational structure of the NFL: teams face each other repeatedly over multiple seasons, and their historical interactions provide valuable context for predicting future match ups.

We will follow a chronological train-validation-test split to respect temporal order. Older games are used for training, while validation and test sets consist of more recent games. This simulates real forecasting conditions and ensures that the model does not learn from future outcomes.

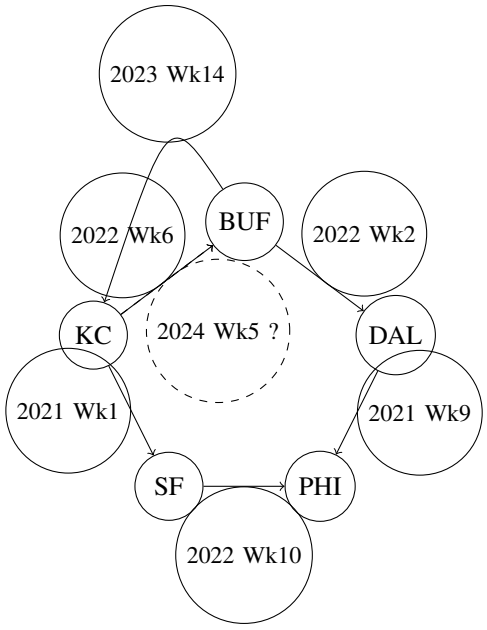

Fig. 2. Example NFL team graph. Nodes represent teams, and edges represent games played. Home team directed toward away team

III. Literature Review

Looking at past methods on this topic, you can see that several traditional models have been used to address the prediction problem. The use of statistical models like Logistic regression and Poisson regression have been used multiple times to estimate game outcomes and score probabilities based on team and player stats and overall performance data. But the downside of using these models is how they oversimplify complex interactions in sports. It fails to adapt to new information or changes, such as a player being injured, players being swapped out, or the team dynamics. It creates it's probability based on a perfect situation, rather than a environment that is always changing.

IV. Method

When trying to address the problems being presented from past sports prediction models, our idea is to use a GNN approach, but more specifically to use the Graph Attention Network architecture. We started with the GAT model, which is great for a situation like this, since the architecture is able to see the use of the relationships and the interactions between teams, players, and games well. And the data of such relationships between teams, players, and games is structured like a graph so well that a GAT model seems to be the perfect choice. In this situation, you can have the nodes being corresponded to each of the entities like teams or players, with the edges being represented like past games, similar performance, or even average end score. With this background, the GAT architecture has the ability to weigh the importance of a select nodes, instead of being stuck with a simple baseline of all nodes being equal. But throughout our process and research into the GAT model, we came across the GRAPHSAGE model, with more promising findings. This version is an altered variant that uses learnable aggregators like mean or max-pooling on sampled subgraphs to enable scalability and inductive learning on large, dynamic graphs. Instead of using every neighbor, which can be slow and memory heavy, it samples a small set of neighbors, then aggregates their features using simple functions like mean or pooling. This aggregated neighbor information is combined with the node's own features to produce a more defined representation. Because GraphSAGE learns how to aggregate features rather than memorizing the whole graph, it can make predictions for new, unseen nodes, making it a great prediction model for changing situations/games. By looking at the advantages and how it able to match to our demands, this why the GraphSAGE is our selected basis for constructing a model to manage multiple edge relationships between distinct nodes.

V. Evaluations

To ensure that the data being used is unbiased, and gives the most performance measurement, we are going to divide the dataset into training, validation, and then finally the test sets. The training set will help in creating and learning the model parameters, while the validation set will help guide the hyperparameter tuning and model selection. Finally, the test set will help as valuable data for the final performance test at the end of the entire process. During this entire process we are also going to monitor any data leakage, as in the past, sports data is time-dependant. Making it more prone to leaking data as past models will confuse present data of teams wins and loses with the numbers from the past.

Dataset split: We are going to run training on past NFL game seasons, Validate with the most recent season, and test it on the newest season Metrics:

- Classification Accuracy
- F1 Score

Results will be displayed in tables and plots, creating the graph that will show the full correlation between the data and the final probability of which team is going to win.

VI. Team Contribution

For this project, it was completed by 3 members who had various responsibilities during the entire process.

- **Kishan:** Will Collect and format the database on the historical NFL games data, and go through changing and creating the datasets needed to link teams, games, and the results of those games. Handling the data, by cleaning or removing missing records as well as helping to integrating model layers.
- **Nick:** Will design and create the idea of the framework, helping to plan out the use, the defining of the nodes, relationships, and edge representations. He will have part in testing the experiment setup and evaluation, providing feedback and improvement to areas that need change.
- **Jerry:** Will research the literature review on past data models and prior work in sports analytics and GNN-based modeling. And finalize the proposal by looking over different parts and verifying consistency in all the sections of our reports.
- **Experiment Setup and Evaluation:** All three members will contribute to this section of the project, with Kishan setting up the foundation and performing the training, validation, and testing based on the sports data. Jerry will work on fixing any issues that arise while reviewing the code from each section, evaluate model performance using metrics like accuracy and F1 score, and generate visualizations to display the final graph. Nick will play an integral role in implementing the Graph Attention Network architecture, focusing on attention mechanisms, edge relationships, and the impact of node weights across different parts of the code.

VII. Related Work

When beginning our research into creating a prediction model for football, we came across a ton of relevant work from other experts who were developing their own machine learning

prediction models based on football or even other sports. The planning and execution of the early research process for these projects are quite identical, with the exception of how the model is taught and reinforced to adjust to new variables that are continuously introduced as these projects move forward. An interesting example would be the work of Ondřej Hubáček, Gustav Šír, and Filip Železný, who wanted to propose a more forecasting system, then prediction analysis of matches [1]. During the construction of their model, the focus was towards using this to take advantage of the sports betting market as a way to produce the most amount of profit [1]. They argue that maximizing predictive accuracy alone, as many prior works did, is useless because even a "good" model may not produce profit if the model's predictions are strongly correlated with the data sets implied probabilities [1]. In order to counteract this, they look for forecasts that systematically deviate from the data sets consensus while maintaining respectable predictive performance [1].

Another example of related work would be the research article from Conor Walsh and Alok Joshi [2]. They focus on the overall accuracy or calibration use cases and which type of foundation is best to build a model. To test this, the authors trained various models on multiple historical data sets from the NBA. Then created two pipelines where one selects the "best" model based on accuracy, and the other is based on calibration [2]. Their results were quite surprising as betting systems using the calibration models had higher returns, with it consistently beating the accuracy model [2].

While these two articles have shown machine learning in unique situations, this third article by Sascha Wilkens and her works in implementing machine learning models to professional men's and women's tennis matches, and assesses whether such models can reliably outperform the betting market [3]. The research is quite interesting as she uses a variety of machine-learning models on match and player specific data, then evaluate their performance [3]. The key finding is that the prediction accuracy, no matter the type of model, rarely exceeds about 70 percent [3]. Even with rich data sets, the models struggle to reach past this ceiling with current machine learning techniques[3]. Overall, these research articles provide crucial data and context for our machine learning model. As they emphasize how difficult it is to anticipate sports matches and the foundation of the model being created. These findings motivate the direction of our own football prediction model, as we chose to focus on building a model that is grounded in realistic expectations then perfect accuracy.

VIII. Method Section: Full Technical Reproducibility

Our model uses learnable team embeddings combined with a two layer GraphSAGE encoder that generates a 64 dimensional representation for each team. Every team in the NFL is assigned a unique integer ID, and each ID maps to a learnable vector size 64. These embeddings act as the initial node features for the graph. The first GraphSAGE layer takes these embeddings and aggregates information from neighboring nodes, where each neighbor represents an opponent from a past game. The output of the first layer is passed through a ReLU activation and then into a second GraphSAGE layer, which produces the final team representations. These representations summarize each team's historical performance and interactions with other teams.

To convert these node embeddings into a prediction for a specific game, we use a small MLP based edge classifier. For each game, we collect the embedding of the home team and the embedding of he away team, and we concatenate them together with the standardized closing odds from the dataset. This combined feature vector is passed through a two layer MLP that outputs a single logit representing the predicted probability that the home team will win. During training, this logit is converted to a probability using the sigmoid function

The entire model is trained end to end using the Adam optimizer with a learning rate of 0.001 and a weight decay of 0.00001. We trained for up to 100 epochs and used early stopping with a patience of 10 epochs to prevent overfitting. A binary cross entropy loss function was used to compare the predicted probabilities with the actual game outcomes. Gradient clipping with a maximum norm of 1.0 was applied to maintain stable training behavior.

To ensure reproducibility, we report the exact tensor shapes used in the model. The input node feature matrix has a shape [num_nodes, 64], since each team is represented by a 64 dimensional embedding vector. The edge index tensor has shape [2, num_edges], where each column identifies the home team and away team for a particular game. The edge feature matrix has shape [num_edges, 2], which corresponds to the home closing odds and away closing odds for that game. The label tensor has shape [num_edges], containing a value of 1 for the home team wins and 0 for away team wins. The output of the model for each edge is a single logit, which can be viewed as a tensor of shape [num_edges, 1].

IX. Dataset & Graph Construction

When talking about how we handled the dataset and the prepossessing it from raw data into a usable form for training. Before starting the process we had to manually clean the data, as the original had missing cells from past games that greatly impacted the overall quality of the data, damaging the training model by having nothing to work with. After getting rid of the empty cells, the data was much cleaner and the model was able to understand and learn from it at a better state. The process starts by loading the columns of information in the excel file like home team, away team, final scores, and closing betting odds. The team names are standardized by removing extra whitespace or formatting inconsistencies, to ensure that each team is represented uniquely. Next, the full set of unique NFL teams is extracted from both the Home Team and Away Team columns. Each team is then assigned a unique integer ID, which allows PyTorch Geometric to reference teams numerically rather than by string. These team IDs are used to replace the team-name columns and ensuring

that all the processing operates on integer tensors. Finally, the numerical game level columns like the scores and odds, are converted to the correct data types, and the dataset is sorted chronologically. This allows train, validation, and test splits to be applied correctly and without leaking information from future games into the past.

The graph construction is made to display the performance of the betting model, with it using a form of fake bets. The algorithm generates three straightforward graphs, image in figure that show you how successfully your model trained and how profitable its predictions were once all the bets have been simulated. The first graph (Figure 3) is generated by graphing the training loss and validation loss you recorded after each epoch, demonstrating how the model learned over time and whether it started to overfit. The second graph (Figure 4) tracks the validation Brier score throughout epochs, which demonstrates how accurate and well-calibrated the model's predicted probabilities grew during training. The final graph (Figure 5) displays your total profit from all of your bets. It does this by plotting your profit after each bet, adding a straight trend line to indicate whether overall profit is rising or falling, and adding a rolling average line to reduce noise. When combined, these charts provide an accurate representation of the model's calibration, performance, and actual betting outcomes.

When looking into graph, we can see that each node is representing one NFL team, giving us a total of 32 nodes in the final graph. Each of those teams are given a unique ID, that keeps them separate from each others. The nodes also contain placeholder features and team-level statistics of performance from the dataset. Each edge in the graph is to represent a single NFL game, it being directed from the home team to the away team.

Every edge has two features: the home team's closing betting odds ad the away team's closing betting odds. Both of the features help capture how the betting market is viewed in each matchup. Each edge's label represents the game's result; a value of 1 implies that the home team won, and a value of 0 implies that the away team did. In order to classify each game, the model learns from the node information, edge structure, edge features, and binary labels, because its goal is to predict game outcomes.

All these processes help in the final train/val/test split. As it performed on the edges in chronological order, so that the model is always learning from past games, which is then used to evaluate on the future games. After sorting all the games by date, the past 60 percent of games are used for training, 20 percent on validation, and the last 20 percent is for the testing. During the training section, the model only receives labels that are related to training, which includes the edge index slices, edge features, and more. Ensuring that its never leaking into the training model, and the validation or test edges are with their respective sets. The validation set is used at the end of each epoch to test how effectively the model adjusts to unknown future games, and the validation loss limits it from stopping early. As if the validation performance

Fig. 3. Training and Validation Loss Graph

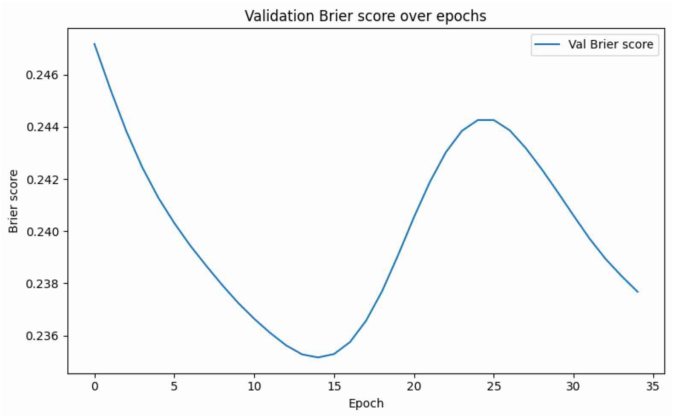

Fig. 4. Performance Over Epoch

stops improving for a defined number of epochs, training ends automatically. In order to provide a fair assessment of how well the final model would perform on genuinely future games, the test set is kept totally unaltered until training and the early halting are finished. Giving us the most accurate data predictions for the entire model that are realistic in any scenario. The data source will be mainly derived from an updated version of the Australia Sports Betting with it more having [5]. There is also a second link that contains code to process the data from its raw state to a more usable form [6].

X. Experiments

To evaluate the performance of our prediction model, we conducted a series of experiments that included model training, validation, probability calibration, and a final betting style backtest. All experiments were carried out on the historical NFL dataset, using a strict temporal split to ensure that the model was always evaluated on games that occurred later in time than those on which it was trained. This prevents data leakage and mimics the real-world setting of predicting future games.

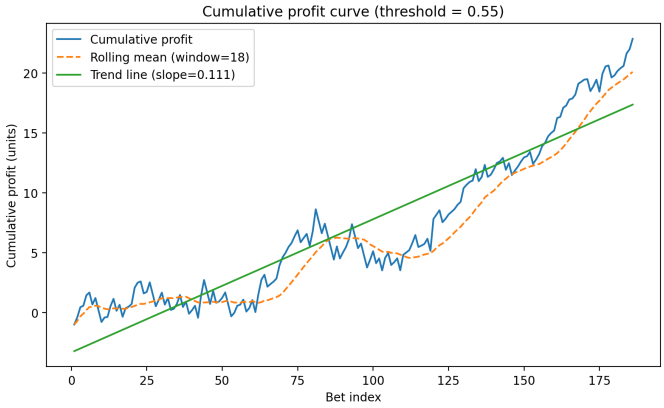

Fig. 5. Total Bets Profits

	Metric	Value
0	Validation Accuracy	0.587600
1	Validation Brier Score	0.237700
2	Test Accuracy	0.630500
3	Test Brier Score	0.217400
4	Backtest (Threshold=0.55) Num Bets	186.000000
5	Total Profit (units)	22.845000
6	ROI per Bet	0.122800
7	Win Rate	0.736600
8	Average EV	-0.012800

Fig. 6. Results Table

Our model uses a two-layer GraphSAGE encoder paired with a small MLP edge-classification head. Each team is represented by a 64-dimensional learnable embedding, and the GraphSAGE layers help the model combine information from previous matches between teams. The MLP then takes the home team embedding, away-team embedding, and closing betting odds and produces a win probability for the home team. We trained the entire model using the Adam optimizer with a learning rate of 0.001 and a small amount of weight decay. Training could run for up to 200 epochs, but we added early stopping with a patience of 20 epochs to prevent overfitting when the validation loss stopped improving.

During training, we tracked metrics such as accuracy, Brier score, and loss on both the training and validation sets. The model reached its best validation accuracy of about 0.5876 and a validation Brier score of 0.2377, and we saved that version of the model for evaluation. Once training was completed, we noticed that the raw output probabilities were somewhat overconfident, which is common with neural networks. To address this, we applied temperature scaling using the validation set. This calibration step made the predicted probabilities more consistent with real-world outcomes, resulting in a learned temperature value of around 1.75.

After calibrating the model, we moved to the part that matters for betting performance. Using the test set, which contains games that were never seen during training, we simulated placing a one-unit bet on the home team whenever the model predicted a win probability above 0.55. This threshold was chosen to avoid making bets on games where the model was uncertain. Across the entire test set, the model placed 186 bets. The model achieved a win rate of about 0.7366, which means roughly 74 percent of the simulated bets were correct. This produced a total profit of 22.845 units and an ROI of about 12.28 percent per bet. We also created a cumulative profit curve, which showed a steady upward trend across the test set. A linear trend line fitted to the profit curve also had a positive slope, indicating that the model was consistently profitable over time.

Overall, the experiments show that our graph-based model can produce well-calibrated predictions and generate positive returns in a simulated betting environment. While there are still many ways the model could be improved, such as adding more detailed team statistics or testing additional GNN architectures, the results demonstrate that a GNN approach has strong potential in sports betting prediction.

XI. Team Contribution Statement

Kishan: Worked on creating the code for the training Loop, as well as researching the work needed for the related work section of the paper. Helped in creating the visualization graphs for the project, as optimizing the final processing of the data by manually getting rid of empty cells that were impacting the training process.

Nick: Contributed written sections of the final report, specifically the Experiments, and the Final Method Section. He also helped identify and debug issues in the code, and assisted with portions of the model implementation. Also helped out in writing the code needed for creating the requirements.txt file.

Jerry: Helped in creating GRAPHSAGE model code implementation, as well as the code for calibration validation and back-testing. Also created the formal results table for visualizations, and the initial online data scraping for the project. As well as creating the code to build nodes set of teams for the model itself.

This contribution statement is a little bit different from the first contribution statement, as we had to initially move around the some job responsibilities to be people who are more knowledge for the given task. This allowed us to complete our work on time and learn from each other on how to work and write the code needed to have the entire model work in the first place.

REFERENCES

[1] O. Hubáček, G. Šourek, and F. Železný, “Exploiting sports-betting market using machine learning,” *International Journal of Forecasting*, vol. 35, no. 2, 2019. :contentReference[oaicite:0]index=0

[2] C. Walsh and A. Joshi, “Machine learning for sports betting: Should model selection be based on accuracy or calibration?,” *Machine Learning with Applications*, vol. 16, 2024, Article 100539. :contentReference[oaicite:1]index=1

[3] S. Wilkens, “Sports prediction and betting models in the machine learning age: The case of tennis,” *Journal of Sports Analytics*, vol. 7, no. 2, pp. 99–117, 2021. :contentReference[oaicite:2]index=2

[4] Australia Sports Betting, ”Historical NFL results and odds data,” *https://www.aussportsbetting.com/data/historical-nfl-results-and-odds-data/*

[5] J. Guerrero, ”NFL Dataset,” *https://docs.google.com/spreadsheets/d/1LObox68UxXi3oQfEsWDqDBN8QNzbVX5W/edit?usp=sharing&ouid=116021263276656483391&rtpof=true&sd=true*

[6] K. Peddi, ”PreProcessing Data,” *https://colab.research.google.com/drive/1qXTu1eaIRXKyUqeqITuaOLfGxCDLUprt?usp=sharing*